Learning Turbulence Closures without Losing the Physics

RANS, inverse PINN, neural regression, and reproducible evidence

FlowMLLab | Week 14 | Ehsan Roohi | Graduate research lecture

1. The question: better k, not just a prettier velocity curve

Research foundation: Lars Davidson's pyCALC-RANS and PINN-NN turbulence-model workflow. DNS reference: Myoungkyu Lee and Robert D. Moser. These lecture explanations and newly generated figures are a FlowMLLab teaching adaptation, not Davidson's original slides.

A mean velocity profile can look convincing while the predicted turbulent kinetic energy is substantially wrong. We will distinguish the observable we want to improve from the equations that constrain it, reproduce source-code operations, and ask what survives feedback through a CFD solver.

Learning outcomes: explain a closure coefficient; reconstruct an inverse-physics target; train the original small neural regression model; separate interpolation from transfer; and report both successful checks and failed convergence criteria.

Suggested format: 75-minute lecture plus a 90-minute computer lab. Prerequisites: Reynolds averaging, k-omega modeling, finite-volume residuals, neural regression and automatic differentiation.

1.1 Definitions, symbols and wall units

RANS: the Reynolds-averaged Navier-Stokes equations, obtained by averaging the flow over turbulent fluctuations; the averaging leaves an unknown Reynolds-stress term that a turbulence model (a closure) must express in terms of the mean flow. k-omega model: a two-equation eddy-viscosity closure that transports the turbulent kinetic energy k and the specific dissipation rate omega and sets the eddy viscosity nu_t = k/omega. DNS: direct numerical simulation, which resolves all turbulent scales and serves here as the reference truth (Lee and Moser, channel flow). PINN: a physics-informed neural network, a network trained by minimizing the residual of a differential equation; here it is used inversely, to infer an unknown coefficient function from reference data, not to solve the flow.

Wall units: u_tau = sqrt(tau_w / rho) is the friction velocity, and Re_tau = u_tau delta / nu the friction Reynolds number, with delta the channel half-height. Distances and velocities are scaled as y+ = y u_tau / nu, U+ = U / u_tau, k+ = k / u_tau^2 and omega+ = omega nu / u_tau^2. In the channel used here delta = u_tau = 1 and nu = 1/5200, so Re_tau = 5200 and y+ = 5200 y. The run names nn5200 and nn10000 refer to Re_tau = 5200 and Re_tau = 10000. At Re_tau = 5200 the viscous sublayer (y+ below about 5) occupies y below 0.001 and the log layer begins near y+ = 30, that is y = 0.006; a stored profile therefore has most of its points very close to the wall, which is why unweighted and cell-width-weighted norms differ.

Closure coefficients: sigma_k (turbulent diffusion of k), C_k (a multiplier on the destruction term 0.09 k omega, so C_k = 1 is the standard model) and C_omega2 (the coefficient of the omega destruction term, 0.075 in the standard model; here an absolute value, not a multiplier). Inverse PINN: the network that infers the diffusion coefficient a(y) from the k balance. Table-based PINN stage: the RANS run that uses the spatial correction tables produced by that inverse step, before any feature-based network is trained. PCHIP: a shape-preserving piecewise-cubic interpolant, used as a non-neural control.

2. What does a turbulence closure actually change?

Reynolds averaging introduces the unresolved stress tensor. An eddy-viscosity closure relates its deviatoric part to the resolved strain. This makes the mean-flow equations solvable, but it does not make the turbulence model exact.

For a steady, fully developed channel, the mean flow depends strongly on the eddy viscosity nu_t = k/omega. Increasing k alone increases nu_t, which changes the mean velocity. A coordinated change to omega can preserve the ratio while allowing a different turbulent-energy level.

$$v_t = k/\omega, \qquad P_k = v_t (dU/dy)^2$$

$$0 = P_k - 0.09 C_k k\omega + \frac{d}{dy}[(v + v_t/\sigma_k) dk/dy] \tag{1}$$

$$0 = \frac{5}{9}\frac{\omega}{k} P_k - C_{\omega 2}\omega^2 + \frac{d}{dy}[(v + v_t/\sigma_\omega) d\omega/dy]$$

The three learned quantities have different jobs: sigma_k controls turbulent diffusion of k; C_k scales its destruction; C_omega2 controls omega destruction. C_omega2 is an absolute coefficient here, not another multiplicative correction to 0.075. The unmodified values are sigma_k = 2, C_k = 1 and C_omega2 = 0.075.

The corrected channel solver also ties sigma_omega to sigma_k through min(2*sigma_k, 2); it is not an independently trained fourth network.

Think before fitting: if the velocity already agrees well but k does not, which equation term should you inspect first? Why is fitting U alone insufficient to identify all three correction functions?

3. Establish the right baseline

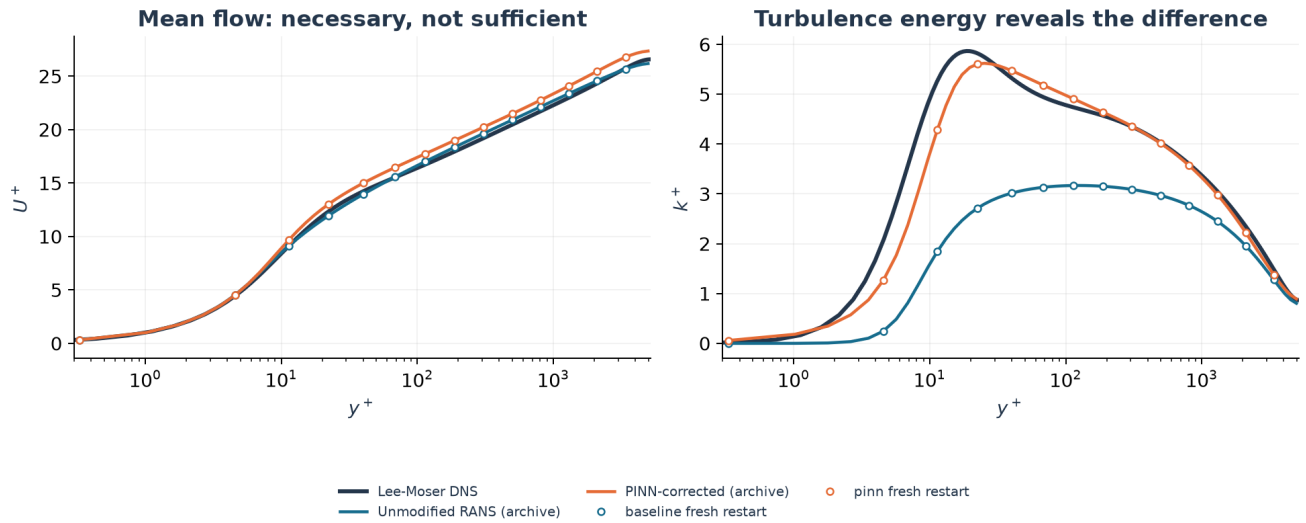


Figure 1. Classical and table-based PINN profiles against Lee-Moser DNS. Markers are fresh restarted solves, not a verified reproduction of the final PINN-NN curves in Figure 8 of the paper.

Use the actual unmodified channel case, not a table that happens to include RANS in its filename. The supplied notebook's embedded profile matches the upstream PINN-corrected directory byte for byte. Calling that profile the classical baseline hides much of the model discrepancy.

The figures distinguish archived source data from freshly executed results. A restarted simulation is a new numerical execution, but not a cold-start demonstration. Record the grid, initial fields, stopping criterion and achieved residual alongside every accuracy claim.

Against the DNS, the unmodified archive profile has a pointwise relative L2 error of 1.7% in U+ and 42.8% in k+; the PINN-corrected archive profile has 5.0% in U+ and 8.9% in k+ (cell-width-weighted values are in the notebook). The correction therefore buys a five-fold improvement in k at the price of a worse mean velocity. These percentages describe the distributed table-based PINN stage, not a verified reproduction of the final PINN-NN curves in the paper's Figure 8. Section 5.1 reports good mean-velocity predictions and improved k for the final model. Do not attribute our particular velocity-error increase to that final-model claim.

All profile norms must identify their weighting. An unweighted Euclidean norm on a stretched grid emphasizes regions with many stored points. A cell-width-weighted norm answers a different question. Neither is an experimental uncertainty estimate.

4. One research pipeline, several evidence levels

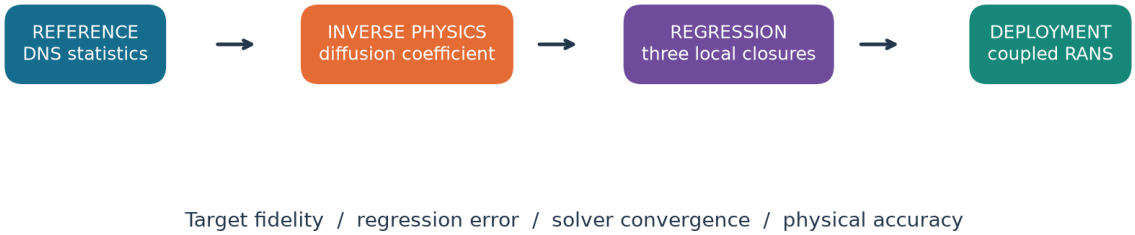


Figure 2. Four stages of the attributed research workflow, each requiring a different kind of evidence.

The inverse problem supplies a spatial diffusion correction using reference statistics. Supervised networks then express the correction coefficients through local flow features. Deployment evaluates those networks inside the iterative RANS calculation.

Do not collapse these stages into one accuracy number. A low PINN residual tests the inverse equation; a low NN regression error tests coefficient approximation; a converged CFD residual tests the numerical solve; agreement with DNS tests selected physical observables.

Each stage can succeed while a later stage fails. A model fitted on a known channel profile can have an excellent random-point test score without establishing transfer to separation, a new geometry or a new Reynolds number.

5. The inverse-physics step

Treat the turbulent diffusion coefficient as an unknown function a(y). With reference production, dissipation and k, the channel energy balance gives a differential constraint on a. A neural representation allows its derivative to be calculated by automatic differentiation.

$$r(y) = a'(y)k'(y) + (\nu + a(y))k''(y) + P_k(y) - \varepsilon(y)$$

$$\mathrm{L} = \sum_i r(y_i)^2 + 1000\mathrm{L}_{BC} \tag{2}$$

$$\sigma_k(y) = \nu_t(y)/a(y)$$

The released implementation combines a summed interior residual with a boundary penalty of 1000. Those numerical weights are implementation choices, not universal physical constants. Derivatives of reference k are calculated numerically before interpolation onto the RANS grid.

Near a stationary point of k, its first derivative is small; inference of a from the balance becomes sensitive. Reference discretization, endpoint conditions, positivity, smoothing and the exact transport terms therefore matter. Small training loss alone cannot certify a uniquely identified physical closure.

The long PINN optimization and archived checkpoints must be labeled separately from a fresh supervised fit. Loading coefficients previously obtained by PINN is not training a PINN during this lab.

Our full 200,000-epoch source run reached a minimum loss near 1.24 but finished near 231. The original script does not select the best checkpoint. Do not substitute the minimum for the final result or claim reproduction of the paper's reported final loss of 4.

6. Three targets and their units

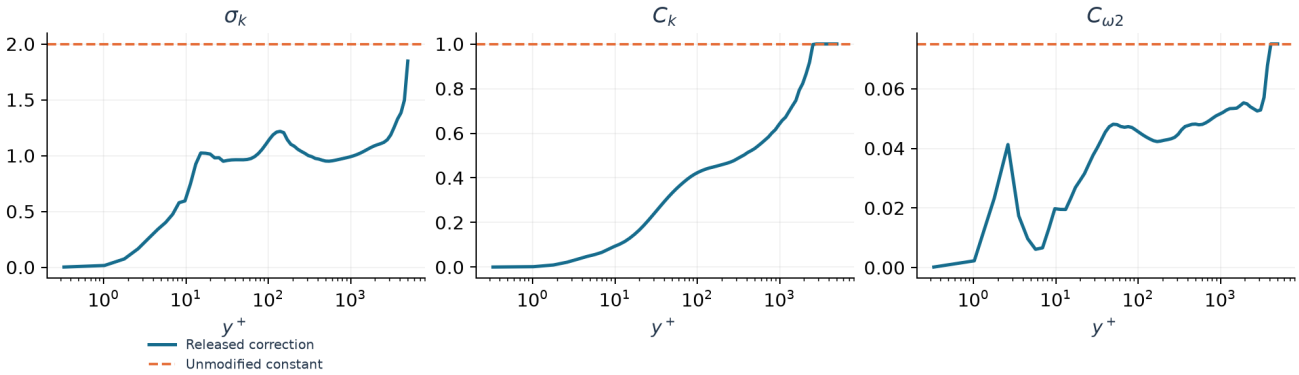


Figure 3. Released spatial correction coefficients and unmodified model constants.

The channel scripts use delta = u_tau = 1 and nu = 1/Re_tau. Thus stored U and k coincide numerically with U+ and k+, but stored omega uses the outer time scale delta/u_tau. To obtain omega+ multiply stored omega by nu/u_tau squared.

At deployment the inputs are nu_t/(y*u_tau) and total shear magnitude divided by u_tau squared. In the training table, u_tau = 1 simplifies these expressions. The released c_k feature calculation caps total shear at 0.995.

The balance-derived targets include caps and local smoothing. These choices must be recorded, not described as exact DNS truth. The k correction and omega correction compensate for changing the diffusion model; they are not independent measurements.

Important execution finding: the released balance script does not regenerate the bundled targets from the bundled inputs. Maximum absolute differences are about 0.525 for the two-column C_k table, 0.0198 for C_omega2 and 0.299 for sigma_k. We preserve both versions and do not claim end-to-end target reproduction.

Lab check: confirm matching coordinates, finite values, target shape and file hashes before training. A matching array length alone does not prove that two profiles share the same grid.

7. Reproduce the network before changing it

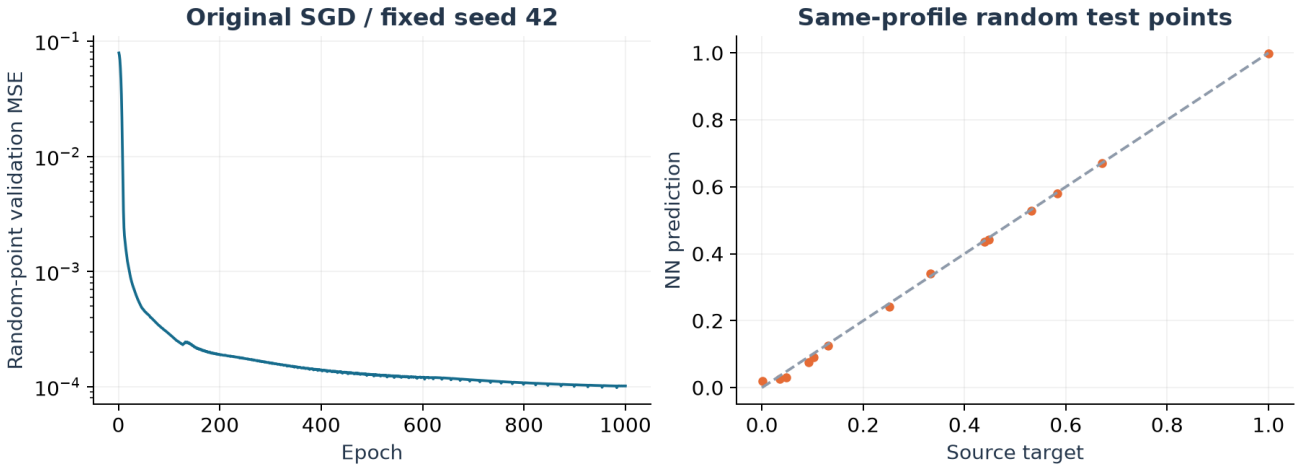


Figure 4. Original c_k regression protocol with an explicit initialization seed. Random-point accuracy is not whole-case transfer.

The released c_k script uses raw local features with MinMax scaling, two hidden layers of ten ReLU units, a linear scalar output and mean-squared error. Training uses SGD, learning rate 0.04, batch size one and 1000 epochs. Its random 80/20 point split uses random_state = 42.

For repeatability the FlowMLLab runner fixes the PyTorch initialization seed and uses one CPU thread. These are explicit orchestration additions. The source creates a learning-rate scheduler but does not step it; reproducing the script therefore means retaining the constant rate.

The original scaler sees the complete profile before the split. Preserve that behavior only in the clearly labeled reproduction. In a new predictive experiment fit preprocessing on development data alone. Never silently improve the protocol and still call the result an exact reproduction.

The historical notebook instead used log-transformed features, standardization, tanh and L-BFGS. That is a legitimate comparison model, but not the original algorithm.

8. The interpolation control

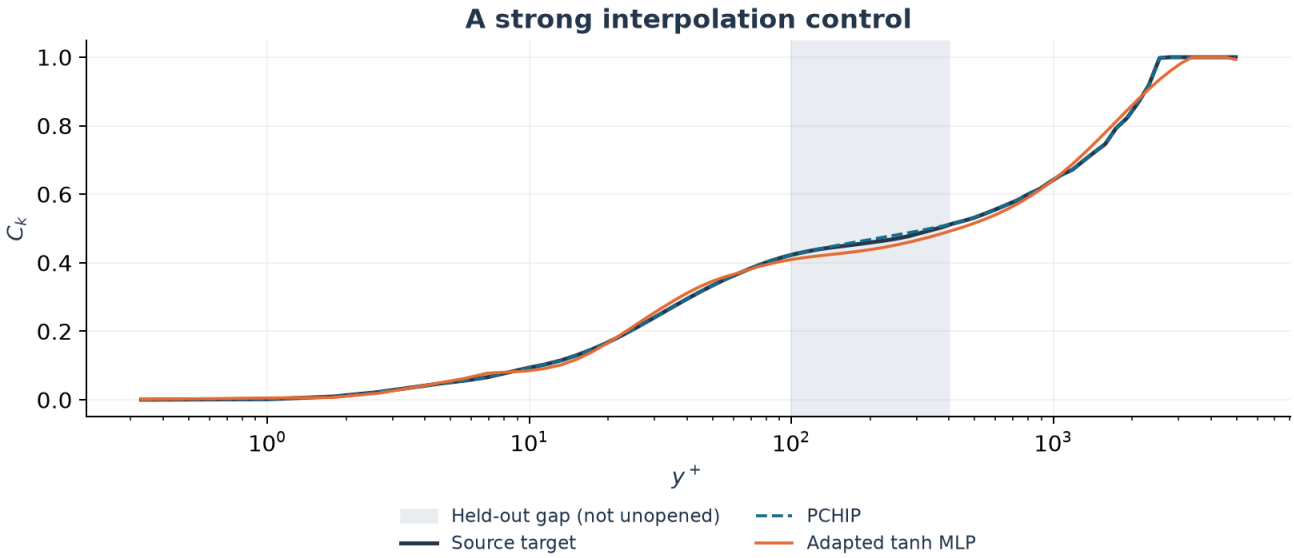


Figure 5. PCHIP and the separately labeled tanh-MLP control over the historically inspected held-out interval.

A contiguous gap in one profile asks whether a model can interpolate across missing wall-normal samples. Compare against shape-preserving PCHIP in log(y+), not only against a constant predictor. PCHIP uses position directly and is intentionally a strong profile-specific control, not a transferable local closure.

The interval 100 <= y+ <= 400 is already part of the inspected teaching material. It is held out from fitting, but no longer an unopened prospective test. Do not rename it blind to make the evidence sound stronger.

Report coefficient errors and the frozen-field destruction diagnostic separately. Multiplying coefficient errors by 0.09*k*omega changes their physical weighting, but does not account for the response of k and omega to a changed closure.

If interpolation wins, retain that result. The educational objective is sound inference, not ensuring that the neural network wins every comparison.

9. Put the closure back into the solver

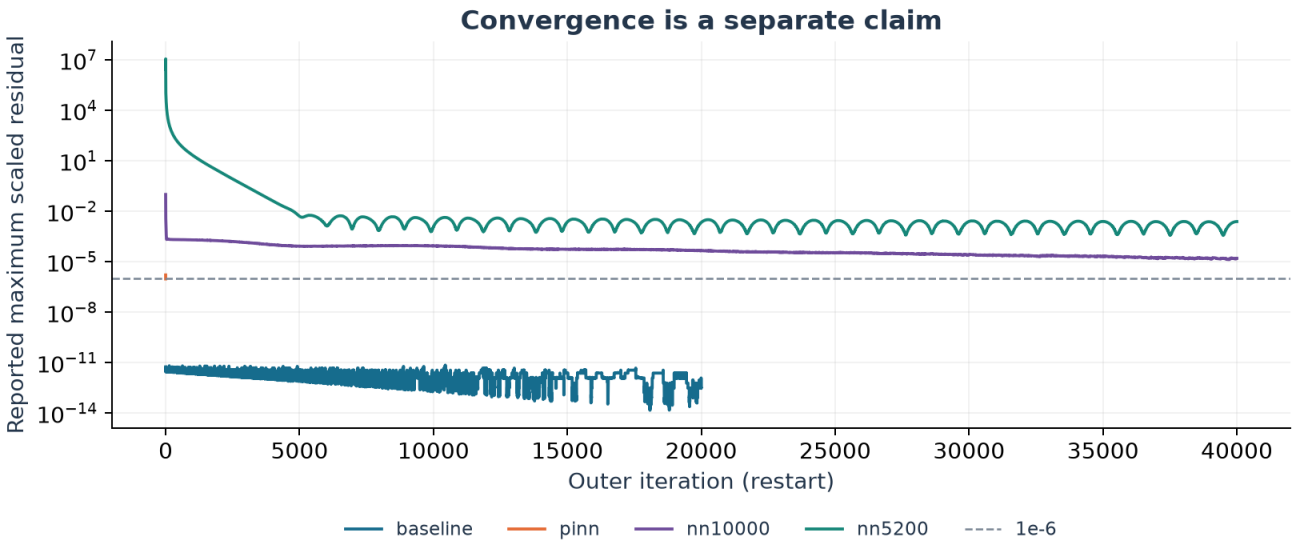


Figure 6. Source-solver residuals. NN5200 is an assembled case; neither NN run meets the requested 1e-6 criterion.

In a coupled run, the current fields produce the features, the networks produce coefficients, and the discretized momentum and turbulence equations update the fields. Clipping and relaxation can help stabilize this feedback, but also change the deployed map.

Check finite fields, positivity, residual trends, wall shear, profile changes and coefficient bounds. For the unit-pressure-gradient half-channel, the expected wall shear is one in the selected normalization. This is an independent physical check, not a substitute for all equation residuals.

The source package includes corrected initialization code but a stale preassembled executable. Rebuilding from the corrected source is essential. The exact source hashes and any compatibility changes belong in the run record.

A residual below an operational tolerance must not be described as satisfying a stricter original tolerance. Equally, a tiny residual does not establish mesh independence or correctness of the turbulence model.

10. A reproducibility ledger

Table 1. Actual iterations, achieved residual and the original stopping gate of each run; the baseline gate (1e-14) is far stricter than the others (1e-6), so its NOT MET at 3e-13 is a tolerance statement, not a sign of a worse solve.

Fresh run	Iterations	Residual	Gate threshold	Original gate
baseline	20000	3.12e-13	1e-14	NOT MET
pinn	2	9.31e-07	1e-06	PASS
nn10000	40000	1.57e-05	1e-06	NOT MET
nn5200	40000	2.39e-03	1e-06	NOT MET

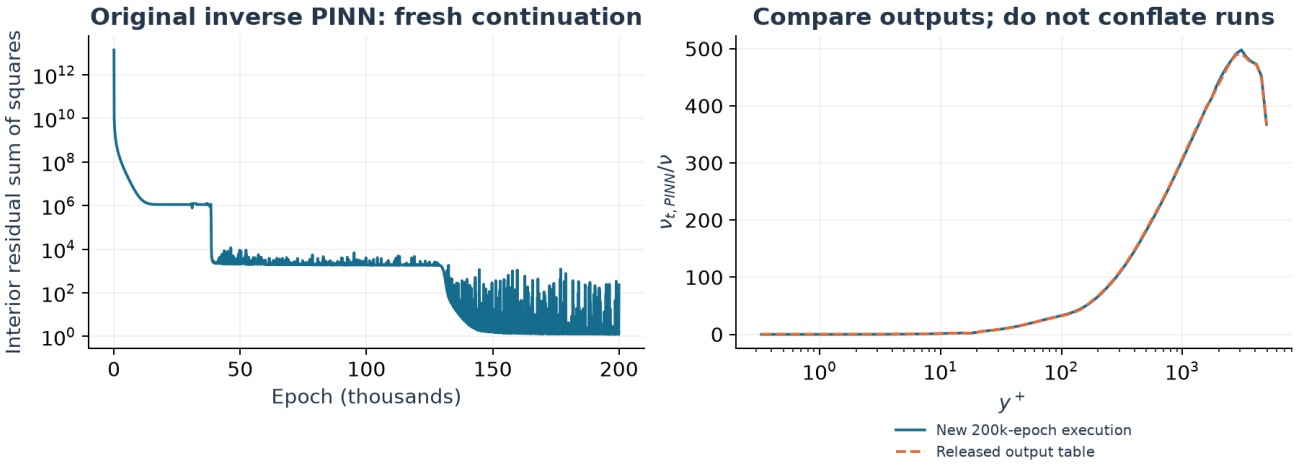


Figure 7. Fresh 200,000-epoch checkpoint continuation and the released diffusion profile. Minimum and final loss are distinct.

For each result keep: source URL and archive hash; input hashes; software versions; seed; grid; restart or cold-start status; actual iterations and stopping criterion; coefficient clipping; and the reference used for each error measure.

The supplied notebook is retained as a historical input and executed separately. The corrected teaching notebook does not overwrite it. Source code, retained author checkpoints, newly trained models and new solver outputs are separate categories.

The instructor reported Davidson's permission in the shared conversation. That statement is not a general license to relicense or distribute the complete source archive. The external code stays in the local download area; cite Davidson for the solver and workflow, and Lee-Moser for DNS.

The corresponding 2026 journal paper is the principal research reading. The older ETMM15 paper is useful historical context, but should not be the only reference for the later PINN-NN pipeline.

11. What this lecture does not prove

Reproduction of a known channel result is not a new discovery and not a prospective generalization study. A restarted run does not establish robustness from arbitrary initial conditions. One mesh does not establish discretization independence.

Changing Reynolds number while using a source-provided trained model can test execution of a published transfer case, but it is not a newly blinded benchmark. Keep the training case, model-development history and evaluation case explicit.

A research extension would freeze the complete preprocessing and model-selection protocol, reserve unopened whole cases, include a classical closure baseline, report failures, and examine uncertainty and out-of-support features. Separated flows require additional evidence; do not infer cavity or airfoil validity from channel interpolation.

Discussion: which is more informative for deployment, a two-percent coefficient error or a stable solve with slightly improved k but degraded wall shear? Explain why the answer depends on the intended use.

12. Assessment and primary references

Student submission: one provenance table; the true-baseline and corrected-flow comparison; original-network and interpolation-control metrics; a convergence and wall-shear check; and a 200-word claim boundary. Identify one implementation detail that would materially change the interpretation if omitted.

Extension exercise: derive the integrated total-shear balance for the half-channel. Compare its prediction with the numerical profile. Explain why differentiation errors and cell-face interpolation can affect a pointwise diagnostic even when the finite-volume balance converges.

L. Davidson, Using Physics Informed Neural Network (PINN) and Neural Network (NN) to Improve a k-omega Turbulence Model, Journal of Turbulence 27(7), 187-208 (2026). DOI: 10.1080/14685248.2026.2665148. Public preprint: arxiv.org/abs/2511.12493v3.

L. Davidson, pyCALC-RANS: A Python Code for Two-Dimensional Turbulent Steady Flow (2026). Report and source distribution: cfd-sweden.se/lada/pyCALC-RANS.html. The PINN-NN distribution page records the September 2026 initialization fixes.

M. Lee and R. D. Moser, Direct numerical simulation of turbulent channel flow up to Re_tau approximately 5200, Journal of Fluid Mechanics 774, 395-415 (2015). DOI: 10.1017/jfm.2015.268.

Worked inverse problem and conditioning

The notebook first identifies the trivial linear warm-up a=0.1+0.2y: the boundary lifting already contains the answer. The required manufactured fit uses a=0.1+0.2y+0.4y(1-y), k=y(1-y), and forcing f=2(nu+a)-a'(1-2y). The zero correction must fail; the trained correction is compared with the analytic nonlinear coefficient on 201 independent points. Expanding d[(nu+a)k']/dy+f=0 yields a'k'+(nu+a)k''+f=0. Integrating yields (nu+a)k'=C-integral(f)dy: when k' approaches zero, dividing noisy flux by k' is ill conditioned. A flux error 0.001 produces coefficient error 0.01 at k'=0.1 but 1 at k'=0.001. Explain how global regularized residual fitting differs from pointwise division, and why diffusion a and destruction correction C_k are distinct inverse quantities.